

ENGAGEMENT BRIEF

Cloud Account Setup & Migration

A defined scope of work for cloud account creation and an existing-project migration to company-owned infrastructure.

CLIENT	ESTIMATED EFFORT	STEPS	WORKING MODE
Hypothetic Pvt Ltd	1 working day	5, sequential	Live screenshare

Engagement Overview

This document is the complete scope of work for a one-off engagement. You are being engaged to create a defined set of cloud and SaaS accounts, transfer an existing source repository, and migrate an existing application's data and deployment to the new accounts.

The work is divided into **five sequential steps**. Each step depends on the previous one. Anything not described in this document is outside the scope of this engagement.

Working Protocol

IMPORTANT All work happens during a **live screenshare session** with the company point of contact. Do not perform any of this work outside the session. The point of contact will be present throughout to enter payment details, approve verification emails, and answer questions.

SECURITY RULES

- Never request, view, or store the company's payment card details. The point of contact enters payment information directly during the screenshare.
- Never email, message, or screenshot any credentials. All credentials are saved directly into a password manager workspace shared with you for this engagement.
- Do not download or save any company files to your personal devices. Use only the shared workspace.
- After the engagement ends, your access to all systems will be revoked. This is standard practice and not a reflection on your work.
- Treat everything you see during the engagement — including domain names, internal email addresses, project details, and infrastructure — as confidential.

Standard Rules for Every Account

These rules apply to *every* account you create. Confirm each one before marking an account complete in the registry.

i Use the role-based email assigned for that step (e.g., `admin@` , `dev@` , `billing@`). Never use a personal email.

ii Generate a unique strong password using the password manager's generator. No password reuse across accounts.

iii Save credentials in the shared password manager vault, with URL, login email, and notes filled in.

iv Enable 2FA / MFA on every account. Store TOTP secrets in the password manager.

v Set company name as **Hypothetic Pvt Ltd** on any billing or account profile field.

vi Set the billing email to `billing@hypothetic.com` .

vii Choose **Mumbai** region (`ap-south-1` or `asia-south1`) wherever the service offers India regions.

viii Document the setup in the shared *Account Registry* page as you go — not at the end.

01 GOOGLE WORKSPACE SETUP

Company domain registered, Google Workspace active, role-based emails working.

- ☐ Open the company's existing Cloudflare Registrar account (the point of contact will provide access). Confirm the company domain is registered. If not yet registered, the point of contact will purchase it during the session.
- ☐ Go to workspace.google.com and start the Business Starter signup. Use the company domain when prompted.
- ☐ When Google asks to verify domain ownership, copy the TXT record value, log in to Cloudflare DNS, and add the TXT record to the company domain.
- ☐ Add the Gmail MX records in Cloudflare DNS using the values Google provides. Wait for verification (usually 1–10 minutes).
- ☐ Create the primary super-admin user (the point of contact will specify the username). Set a strong password generated in the password manager and save it.
- ☐ Enable 2FA on the super-admin account. Save backup codes in the password manager.
- ☐ In Google Admin Console, create the following email aliases (or groups) routing to the super-admin: `admin@` , `dev@` , `billing@` , `security@` , `founders@` .
- ☐ Test by sending an email to each alias from an external account and confirming delivery.
- ☐ Configure SPF, DKIM, and DMARC records in Cloudflare DNS per Google's recommendations. Verify in the Workspace admin console that all three are passing.
- ☐ Update the Account Registry with: Workspace admin URL, super-admin email, plan tier, all aliases created, and 2FA status.

02 VERCEL TEAM SETUP

A new Vercel Team created using the Workspace email — empty, ready to receive the migrated project.

NOTE Use an **incognito browser window** for this step. This prevents any existing personal Vercel session from interfering with the new team setup.

- ☐ Open an incognito window. Go to vercel.com and click *Sign Up*.
- ☐ Choose *Continue with Google* and sign in with `dev@` (the company domain email created in Step 1).
- ☐ When prompted to choose an account type, create a **Team** (not a personal Hobby account). Name it the company name.
- ☐ Skip the project import step — the project will be added in Step 5.
- ☐ Go to *Team Settings* → *General*. Confirm the team URL slug and verify the team email is the Workspace email.
- ☐ Go to *Team Settings* → *Billing*. The point of contact will enter the corporate card directly. Set the billing email to `billing@hypothetic.com`.
- ☐ Go to *Team Settings* → *Security*. Enable the *Require 2FA* setting for all members.
- ☐ Go to *Account Settings* → *Authentication* and enable 2FA on the Vercel user account itself.
- ☐ Update the Account Registry with: Team URL, team slug, login email, plan tier, and 2FA status.

03 GITHUB ORGANIZATION SETUP

Company-owned GitHub Organization ready to receive the migrated repository.

NOTE This step happens before the Supabase setup so that when Vercel and Supabase are connected in later steps, they connect directly to the new GitHub organization rather than to a personal account.

- ☐ In an incognito window, go to github.com and sign up using the `dev@` email. This creates a new GitHub user tied to the Workspace email.
- ☐ Verify the email by clicking the link GitHub sends.
- ☐ Enable 2FA on the new GitHub user (*Settings → Password and authentication*). Use an authenticator app via the password manager. Save the recovery codes.
- ☐ Create a new Organization at github.com/organizations/new. Use the company name as the org name. Choose the Free plan to start.
- ☐ In *Organization Settings → Billing & plans*, set the billing email to `billing@hypothetic.com`.
- ☐ In *Organization Settings → Member privileges*, set base permissions for new members to **Read**. Disable creation of public repositories by members.
- ☐ In *Organization Settings → Authentication security*, require 2FA for all members.
- ☐ If the point of contact provides a second owner account, invite that user as an Owner of the organization.
- ☐ Update the Account Registry with: organization URL, owner email, plan tier, and 2FA enforcement status.

04 SUPABASE ORGANIZATION SETUP

A new Supabase Organization and an empty project, in the Mumbai region, ready to receive migrated data.

- ☐ In an incognito window, go to supabase.com and sign up using `dev@` (the Workspace email). Use *Continue with Google* for cleanest authentication.
- ☐ Verify the email if prompted.
- ☐ Enable 2FA in *Account Settings* → *Security*.
- ☐ Create a new Organization. Use the company name. Set the billing email to `billing@hypothetic.com`.
- ☐ Inside the new organization, create a new project. Project name will be provided by the point of contact.
- ☐ During project creation, set the region to **Mumbai (ap-south-1)** and choose the Free pricing plan.
- ☐ Generate the database password using the password manager. Save it immediately — it cannot be retrieved later from Supabase.
- ☐ Wait for the project to provision (about 2 minutes).
- ☐ Once provisioned, save the project URL, both API keys, and the database connection string to the password manager. The point of contact will use these in Step 5.
- ☐ Update the Account Registry with: organization URL, project name, project reference ID, region, plan tier, and 2FA status.

05 MIGRATION

Existing repository, database, and deployment moved to the newly created accounts — with the live domain cut over only after end-to-end verification.

PLAN A WINDOW Step 5 should run in an uninterrupted block of **2–3 hours** during low-traffic hours. The point of contact will confirm the timing and notify any users in advance if needed.

Order of operations

- **5a** — Migrate the GitHub repository (low risk, easy to revert).
- **5b** — Migrate Supabase data (database, auth, storage).
- **5c** — Set up the Vercel project against the new repo and new Supabase, verify on the temporary Vercel URL, then cut DNS.

5a • Repository migration

Migrate the GitHub repository

- ☐ The point of contact will log in to the existing personal GitHub account that owns the repository. Go to the repo's *Settings* → *Transfer ownership*.
- ☐ Type the repository name to confirm and enter the new organization (created in Step 3) as the destination.
- ☐ In the new organization (signed in as the org owner), accept the incoming transfer.
- ☐ Confirm the repository now appears at `github.com/<org>/<repo>` with all history, issues, PRs, and stars preserved.
- ☐ In the new repo's *Settings* → *Secrets and variables* → *Actions*, confirm all secrets transferred. Note any that did not — these need to be re-added by the point of contact.
- ☐ In *Settings* → *Webhooks*, audit each webhook. Most transfer cleanly, but verify each one points where expected.
- ☐ In *Settings* → *Branches*, re-apply branch protection rules if any. Default: protect `main`, require PR reviews, require status checks to pass.
- ☐ If the original repo had collaborators, re-invite them as members of the new organization with appropriate repo access.
- ☐ Update the local development remote (the point of contact will run this on their machine):

```
# Update the git remote on the local working copy
git remote set-url origin https://github.com/<org>/<repo>.git
git remote -v # Confirm the new URL
git fetch origin
```

5b · Database migration

Migrate Supabase data

This sub-step has three parts: **database**, **auth users**, and **storage**. Run them in order. Verify each part before moving to the next.

DATABASE

- ☐ The point of contact will provide the connection string for the existing (source) Supabase project, just-in-time. Do not store it.
- ☐ Confirm `pg_dump` and `psql` are installed on the working machine. Use PostgreSQL client tools version 15 or higher.
- ☐ Take a fresh dump of the source database to a temporary file:

```
# Dump the source database (schema + data, no owner/privileges)
pg_dump \
  --no-owner \
  --no-privileges \
  --schema=public \
  --schema=auth \
  --schema=storage \
  --file=/tmp/migration-dump.sql \
  "<SOURCE_DB_URL>"
```

- ☐ Inspect the dump file size and a sample of its contents to confirm tables and rows are present.
- ☐ Restore the dump into the new (destination) Supabase project:

```
# Restore into the destination database
psql "<DESTINATION_DB_URL>" -f /tmp/migration-dump.sql
```

- ☐ Run validation queries on the destination: row counts per major table should match the source. The point of contact will provide a list of tables to verify.
- ☐ In the Supabase dashboard for the new project, go to *Authentication* → *Policies* and confirm RLS policies were imported correctly.
- ☐ Securely delete the local dump file:

```
shred -u /tmp/migration-dump.sql # Linux/macOS
```

AUTH USERS

- ☐ The auth schema is included in the database dump above, but verify users exist in the new project's *Authentication* → *Users* dashboard.

- ☐ If users are missing or password hashes need refreshing, follow Supabase's official auth migration guide. The point of contact will be present to make any decisions.
- ☐ Confirm at least one test user can sign in to the new project (the point of contact will perform the login test).

STORAGE

- ☐ List the storage buckets in the source project's *Storage* dashboard.
- ☐ In the destination project, recreate each bucket with the same name and the same public/private setting.
- ☐ Use the Supabase CLI or a custom script to copy objects from source to destination buckets. The point of contact will provide the migration script.
- ☐ Spot-check a few files in the destination buckets to confirm they downloaded correctly.
- ☐ Re-apply storage policies (RLS) on the destination buckets to match the source.

EDGE FUNCTIONS If the source project has Supabase Edge Functions, redeploy them to the new project from the codebase using `supabase functions deploy`. Edge function secrets are not migrated by the database dump and must be re-set on the new project.

5c · Deployment migration

Set up Vercel project and migrate the domain

Vercel projects cannot be transferred between teams directly. The approach is: import the new GitHub repo as a fresh project in the new Vercel team, configure environment variables to point at the new Supabase, verify on the auto-generated `.vercel.app` URL, then cut the custom domain over via DNS.

- ☐ In the new Vercel team (Step 2), click *Add New* → *Project*. Choose *Import Git Repository*.
- ☐ Install the Vercel GitHub app on the new GitHub organization when prompted. Grant access to the migrated repository.
- ☐ Select the migrated repository to import.
- ☐ Configure build settings to match the existing project: framework preset, build command, output directory, install command, and Node version. The point of contact will provide these values.
- ☐ Add environment variables. The point of contact will dictate each variable in the screenshare. Pay special attention: variables that point at Supabase (URL, anon key, service role key) must use the **new** Supabase project's values, not the old ones.
- ☐ Set the production branch (typically `main`) and deploy.

- ☐ Wait for the deployment to complete. Note any build errors and resolve them with the point of contact.
- ☐ Use the auto-generated `.vercel.app` URL to test the deployment end-to-end. The point of contact runs through a verification checklist: home page loads, authentication works, a key user flow completes, data reads/writes succeed.

VERIFY BEFORE CUTOVER Do **not** proceed to the domain cutover step until the point of contact has confirmed the new deployment works correctly on its temporary `.vercel.app` URL. Catching problems before DNS changes is much easier than after.

DOMAIN CUTOVER

- ☐ In the new Vercel project, go to *Settings* → *Domains*. Add the company's custom production domain.
- ☐ Vercel will display the DNS records required (typically a CNAME or A record). Note them carefully.
- ☐ In the **old** Vercel project's *Settings* → *Domains*, remove the custom domain. This releases the domain so the new project can claim it.
- ☐ In Cloudflare DNS, update the existing DNS records to point at the new Vercel project per the values from the previous checkbox. Set proxy status (orange/grey cloud) per the point of contact's instruction.
- ☐ Wait for DNS propagation (usually 1–10 minutes). Refresh the live site URL.
- ☐ Re-run the verification checklist on the live custom domain. Confirm the SSL certificate is provisioned correctly by Vercel.
- ☐ In Vercel, ensure the production deployment is correctly aliased to the custom domain.

POST-CUTOVER

- ☐ Audit any third-party integrations that reference the deployment URL: payment processor webhooks, OAuth callback URLs, transactional email links, monitoring services. Update each one with the new URL or confirm it already uses the custom domain. The point of contact will provide the list.
- ☐ Update the Account Registry with the new project URLs and confirm migration completion.
- ☐ Do **not** delete the old Vercel project, old Supabase project, or old GitHub references during this engagement. The point of contact will keep them for a 30-day observation window before tearing down.

Acceptance Criteria

The engagement is complete when all of the following are verified by the point of contact:

- All accounts (Workspace, Vercel, GitHub, Supabase) created and accessible by the point of contact via the company emails.
- 2FA enabled on every account; recovery codes saved in the password manager.

- Repository fully transferred to the new GitHub organization with history, issues, PRs, and branch protection intact.
- Database row counts on the new Supabase project match the source for all key tables.
- At least one test user can sign in successfully on the new deployment.
- Storage buckets recreated and objects migrated; RLS policies re-applied.
- The live custom domain serves the new Vercel deployment with a valid SSL certificate.
- The Account Registry is fully populated for every account.
- A short hand-off note is sent to the point of contact summarising any pending follow-ups (verification emails not yet clicked, integrations still pointing at old URLs, etc.).

Confidentiality

Everything you see during this engagement — including the company name, domain, internal email addresses, application details, customer data, billing details, and any communications — is confidential. Do not discuss or share any of this information with anyone outside the engagement, on social media, or in any portfolio or testimonial.

By proceeding with this engagement, you acknowledge and agree to these confidentiality terms.

Hypothetic Pvt Ltd

This document is the complete scope of work for this engagement. If anything is unclear, raise it with the point of contact *before* the working session begins. Mid-session questions are welcome and expected.